5.6 寻址方式——操作数寻址

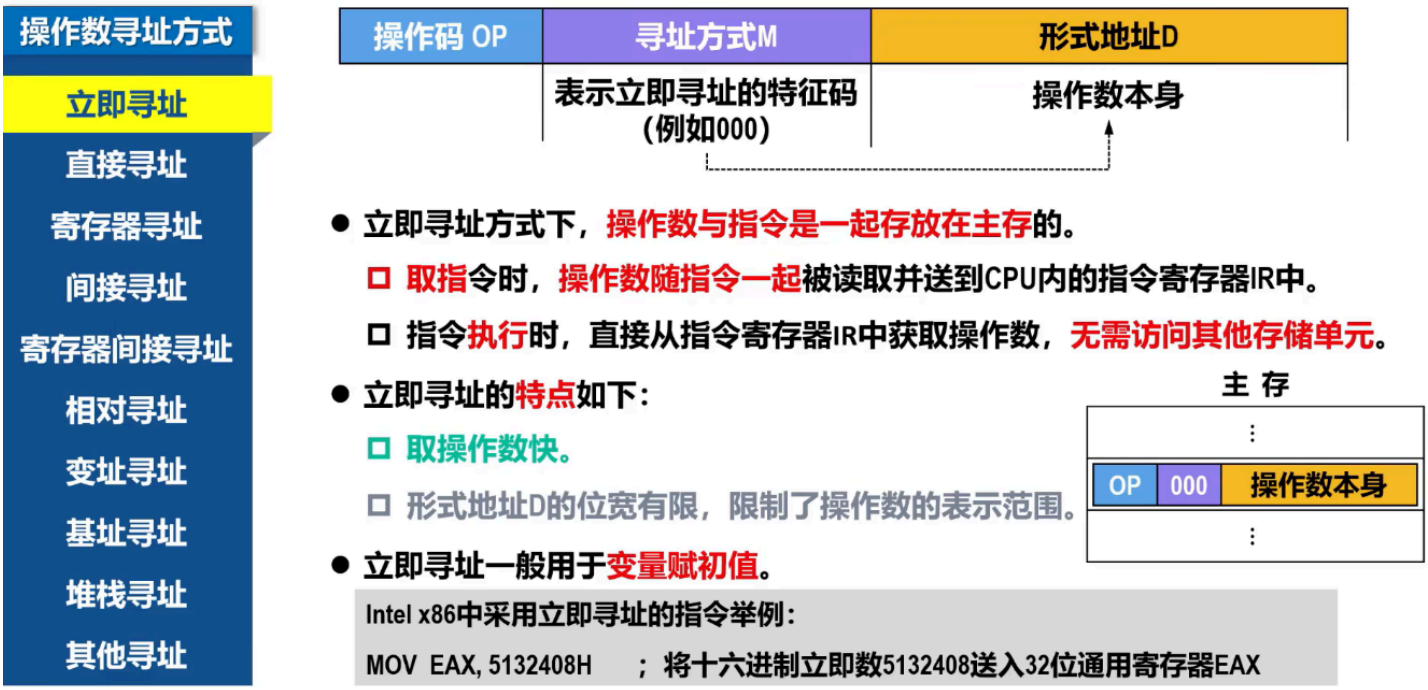
考纲定位：计算机组成原理 · 第 5 章 指令系统（5.6）｜重要度 ★★★★★｜常见考法：选择题/大题（EA 计算、访存次数、相对/变址/基址辨析、复合寻址、数组访问）

一句话核心：所有操作数寻址都在回答两件事——EA 怎么算、要访问几次主存

图旁说明：课件图保留为教学主体；每种寻址只给“本质 / EA / 操作数访存次数（不含取指）/ 易错”四行卡，不重复图内定义与例子

一、操作数寻址方式

1.1 立即寻址



四行卡

- **本质**：操作数直接写在指令中，取指即取数
- **EA**：不存在，形式地址 D 就是操作数本身
- **访存次数**：0（操作数随指令取出，不含取指）
- **易错**：立即数位数受 D 字段限制；立即数不能作目的操作数；立即数按补码还是无符号解释，要看题目约定

1.2 直接寻址





- **直接寻址的特点如下：**
  - 操作数的地址直观，无需专门计算操作数的地址。
  - 形式地址D的位宽有限，限制了操作数的寻址范围。
  - 操作数的地址位于指令中，因此不方便修改。

Intel x86中采用直接寻址的指令举例：

```
MOV  EAX, dword ptr [5132408H]  ; 从主存地址5132408H处；
                                ; 读取一个32位的操作数，
                                ; 并将该操作数送入32位
                                ; 通用寄存器EAX
```



四行卡

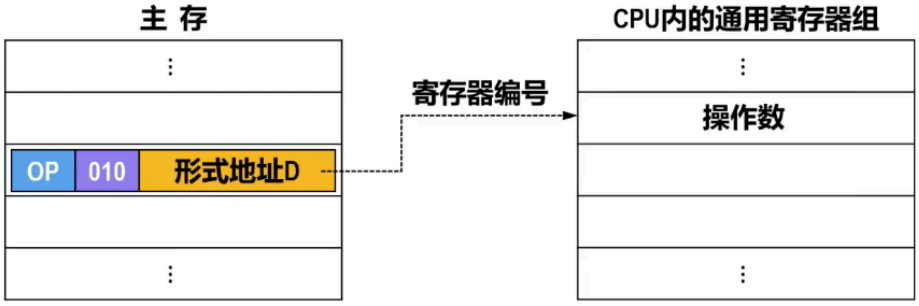
- **本质**：形式地址 D 就是操作数在主存中的地址
- **EA**：$EA = D$
- **访存次数**：1（取操作数）
- **易错**：D 的位数限制寻址范围；改地址必须改指令；直接寻址的“地址”不是寄存器编号

1.3 寄存器寻址





- **寄存器寻址的特点如下：**
  - 获取操作数不需要访问主存（从寄存器获取），指令执行速度快。
  - 由于通用寄存器的数量少，因此所需的形式地址D的位数就少，有利于缩短指令字长，节省存储空间。



四行卡

- **本质**：形式地址 D 是寄存器编号，操作数在寄存器中

- **EA**：寄存器编号 R_D，操作数 $= (R_D)$
- **访存次数**：0（不访问主存）
- **易错**：EA 是寄存器编号，不是寄存器内容；寄存器数量有限，不能为大量操作数提供存储

补充

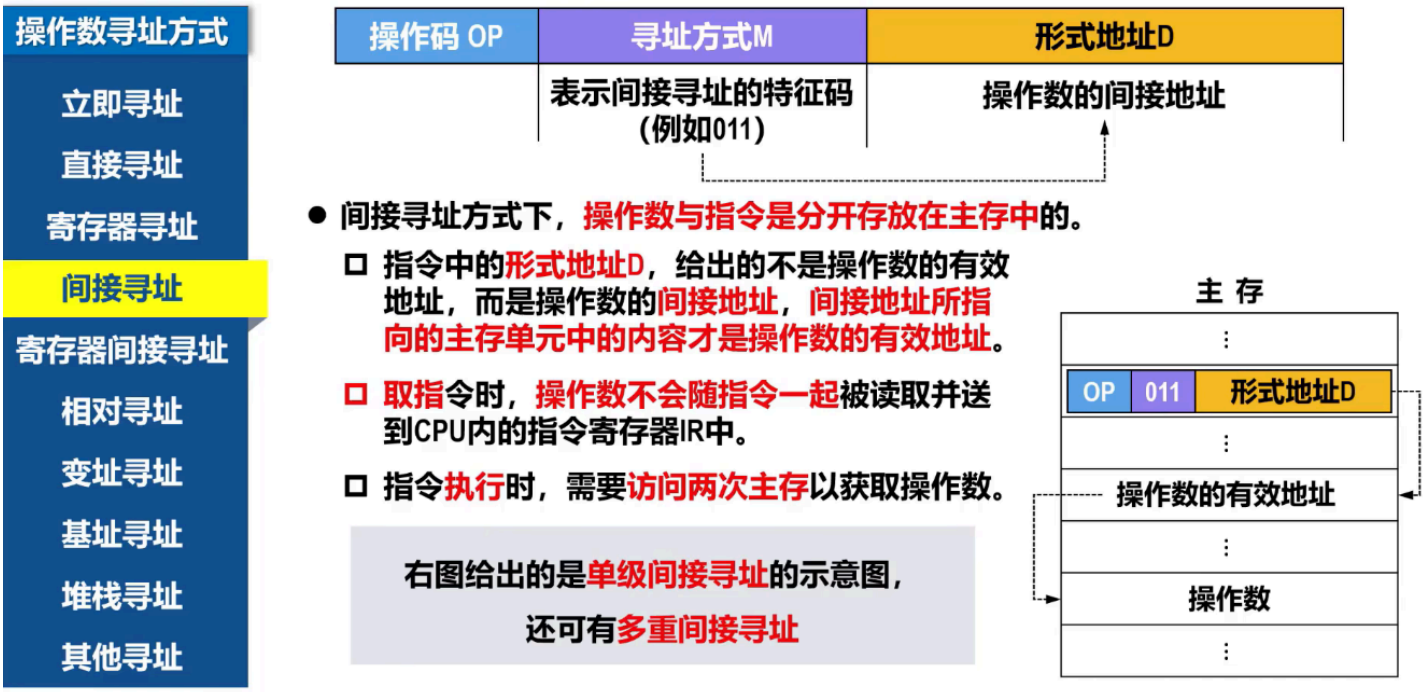
□ 由于CPU内的寄存器数量有限，因此寄存器寻址不能为大量的操作数提供存储服务。

Intel x86中采用寄存器寻址的指令举例：

MOV EAX, EBX ；将32位通用寄存器EBX中的内容送入32位通用寄存器EAX

- 寄存器寻址指令短、执行快，但寄存器数量有限，大量数据仍需放回主存
- 例：`MOV EAX, EBX` 把 EBX 内容送入 EAX，两个操作数都在寄存器

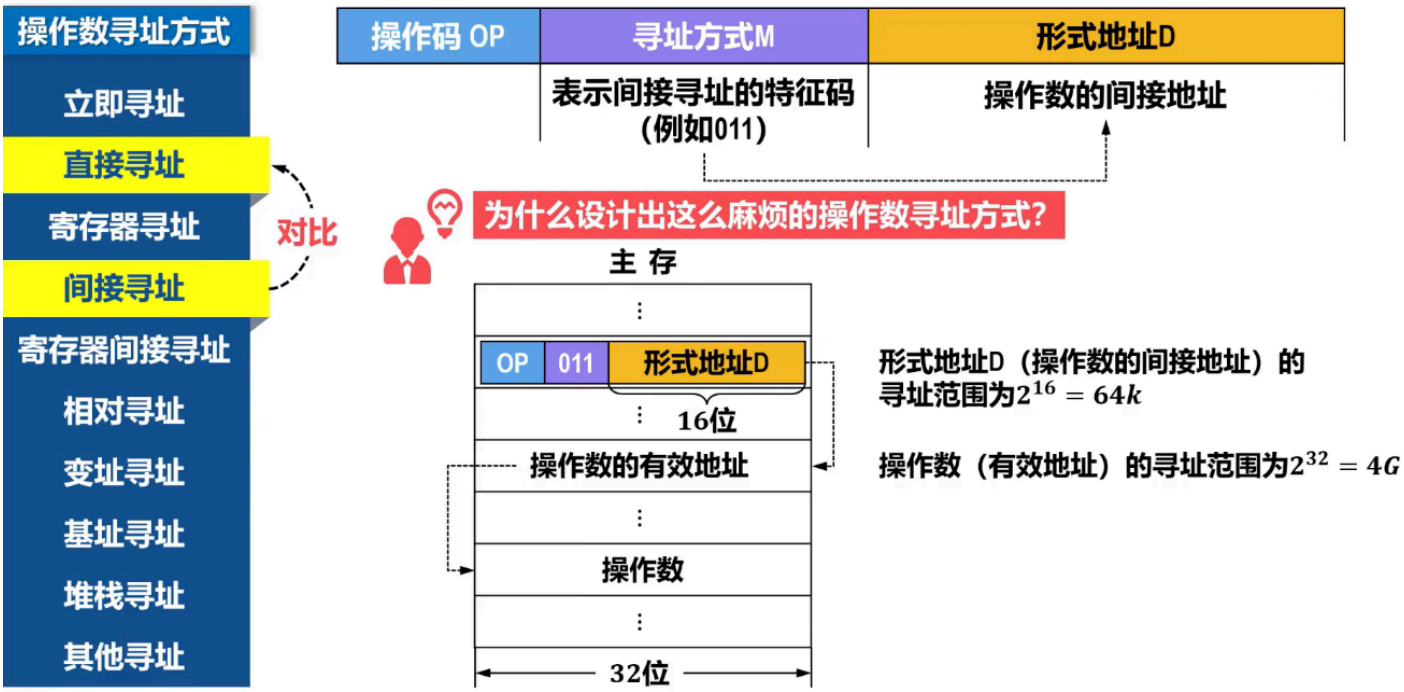
1.4 间接寻址



四行卡

- **本质**：D 指向主存单元，该单元存放操作数的地址
- **EA**：$EA = (D)$，操作数 $= ((D))$
- **访存次数**：2（取地址 1 次 + 取操作数 1 次）；多级间接每多一级再加 1 次
- **易错**：D 是间接地址，不是 EA；间接寻址可扩大寻址范围；多次间接要多次访存

为什么用间接寻址



- 形式地址 D 只有 16 位时，可直接寻址 $2^{16}=64K$；但 D 指向的主存单元可存 32 位地址，操作数寻址范围扩大到 $2^{32}=4G$
- 改操作数地址时，只需改主存中的地址单元，不必改指令

特点及操作举例

- **间接寻址的特点如下：**
  - 相较于直接寻址，能用较短的形式地址D访问较大的主存空间。
  - 相较于直接寻址，当需要修改操作数地址时不再需要改变指令中的形式地址D，只需要修改形式地址D指向的主存单元（存有操作数地址）即可。
  - 指令执行时，需要访问两次主存以获取操作数，降低了指令的执行速度。

某计算机架构中采用间接寻址的指令举例：

```
MOV  EAX, @5132408H     ;从主存地址5132408H所指向的存储单元中取出操作数的
                        ;有效地址，从该地址所指向的存储单元中取出操作数，
                        ;最后送入32位寄存器EAX;
```

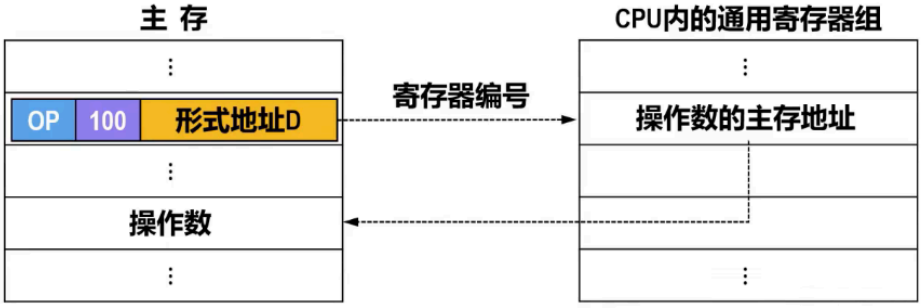
- 优点：扩大寻址范围；便于修改地址
- 缺点：访存次数多，执行速度慢
- 例：`MOV EAX, @5132408H` 先从主存取操作数地址，再按该地址取操作数

1.5 寄存器间接寻址





- 指令中的**形式地址D**，给出的不是操作数的有效地址，而是**通用寄存器的编号**，而**该编号的通用寄存器，其内容就是操作数的主存地址**。
- **取指**令时，**操作数不会随指令一起**被读取并送到CPU内的指令寄存器IR中。
- 指令**执行**时，需要**访问一次主存**以获取操作数。



四行卡

- **本质**：D 是寄存器编号，寄存器内容才是操作数的主存地址
- **EA**：$EA = (R_D)$，操作数 $= ((R_D))$
- **访存次数**：1（取操作数）
- **易错**：与寄存器寻址只差“寄存器内容是否当地址”；与间接寻址只差“地址存在寄存器还是主存”

特点及操作举例

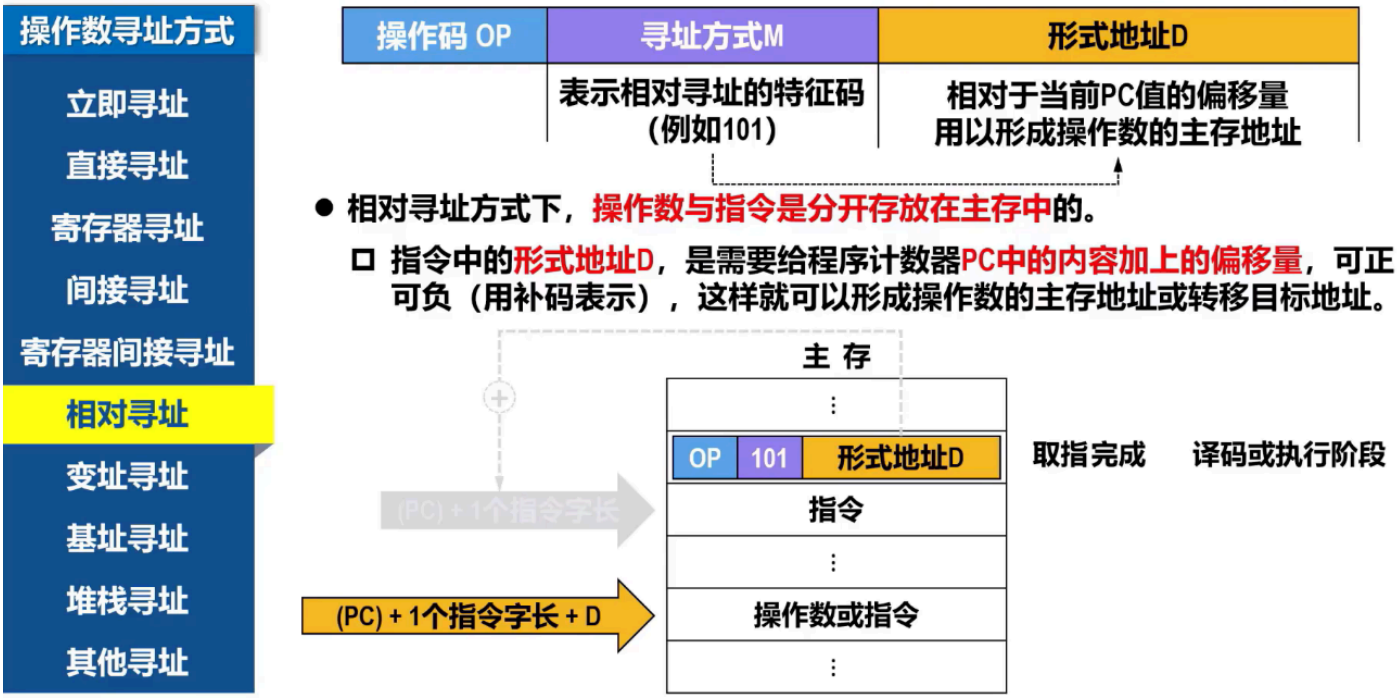
- **寄存器间接寻址的特点如下：**
  - **具有间接寻址的优点（扩充了操作数的寻址范围，提高了编程灵活性），还比间接寻址少访问一次主存。**
  - **与寄存器寻址的缺点类似，由于CPU内的寄存器数量有限，因此寄存器间接寻址不能为大量的操作数提供存储其主存地址的服务。**

Intel x86中采用寄存器间接寻址的指令举例：

```
MOV EAX,[EBX] ；将32位通用寄存器EBX中存储的主存地址所指向主存单元中的内容，
              ；送入32位通用寄存器EAX
```

- 兼具间接寻址扩大范围、提高灵活性的优点，又比间接寻址少访问一次主存
- 缺点与寄存器寻址类似：寄存器数量有限，不能为大量操作数保存主存地址
- 例：`MOV EAX, [EBX]` 把 EBX 中的地址所指主存单元的内容送入 EAX

1.6 相对寻址



四行卡

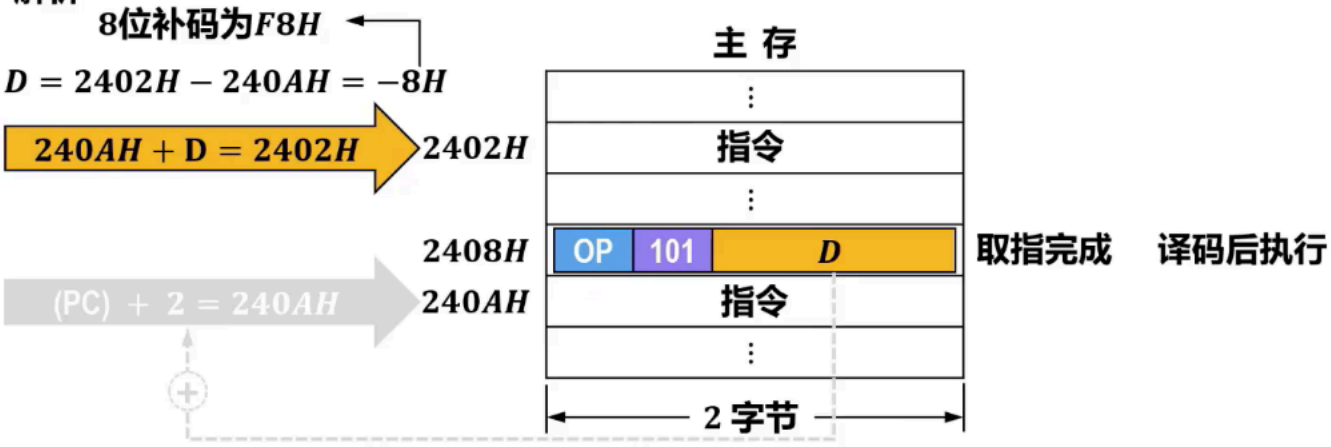
- **本质**：以 PC 为基准加位移，适合位置无关代码和转移指令
- **EA**：$EA = (PC) + D$，其中 PC 是取指后已更新的 PC，D 是补码位移
- **访存次数**：1（取操作数；若用于转移目标，则不取操作数，只改 PC）
- **易错**：PC 基准到底是“当前指令地址”还是“下一条指令地址”；位移可正可负；相对寻址既可寻操作数也可定转移目标

举例



【习题】假设某计算机采用定长指令系统，指令字长为16位，其中形式地址D字段为8位，主存按字节编址。若当前指令的主存地址为$2408H$，采用相对寻址，而转移后的的主存地址为$2402H$，则该相对寻址转移指令的形式地址D字段的值为多少?

解析



- 指令在 2408H，指令长 2 字节，取指后 PC = 240AH
- 目标地址 2402H，位移 $D = 2402H - 240AH = -8H$
- 8 位补码表示：$-8H = F8H$
- 易错：F8H 不是 $+248$，而是 -8；先按补码求真值，再代数

特点



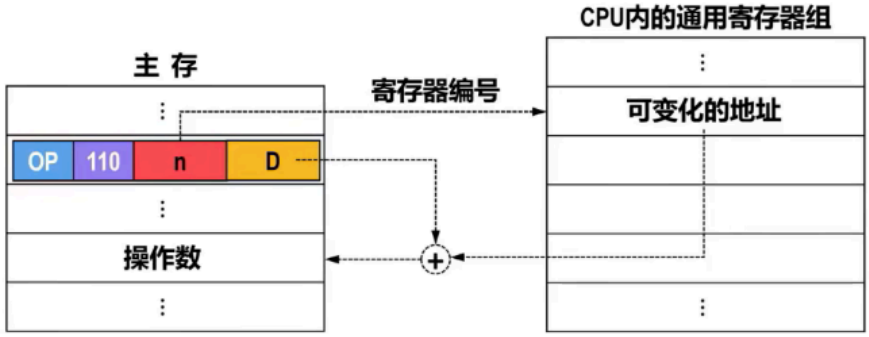
- 相对寻址方式下，操作数与指令是分开存放在主存中的。
  - 指令中的形式地址D，是需要给程序计数器PC中的内容加上的偏移量，可正可负（用补码表示），这样就可以形成操作数的主存地址或转移目标地址。
  - 取指令时，操作数不会随指令一起被读取并送到CPU内的指令寄存器IR中。
  - 指令译码或执行阶段，会计算操作数的主存地址或转移目标地址。
- 相对寻址的特点和用途如下：
  - 程序设计时只需给定指令与操作数之间的相对距离，而不必关心操作数在主存中的绝对地址，可用于实现共享代码的浮动。
  - 实现程序跳转执行。

- 程序员只需给出指令与操作数的相对距离，不必关心绝对地址
- 便于实现共享代码的浮动装入和程序跳转
- 转移目标计算：$目标 = (PC) + D$，其中 PC 为取指后的值

1.7 变址寻址



- 变址寻址方式下，操作数与指令是分开存放在主存中的。
  - 指令中的寄存器编号n，指明使用CPU内部哪个寄存器（通用寄存器或专用寄存器），来存放变化的地址，该寄存器称为变址寄存器，其内部存放的地址与形式地址D相加，即可得到操作数的主存地址。
  - 取指令时，操作数不会随指令一起被读取并送到CPU内的指令寄存器IR中。
  - 指令执行时，需要访问一次主存以获取操作数。



四行卡

- **本质**：形式地址 D 作基准，变址寄存器 IX 的内容可变，适合数组遍历
- **EA**：$EA = (IX) + D$
- **访存次数**：1（取操作数）
- **易错**：IX 的内容由用户程序改变，D 在指令执行过程中保持不变；与基址寻址的“谁变谁不变”正好相反

特点

- 变址寻址的特点和用途如下：
  - 变址寄存器n的内容可变，而形式地址D的值设定后在指令执行过程中将保持不变。变址寄存器n的位数大于形式地址D的位数，扩大了寻址范围（可表示整个存储空间）。
  - 主要应用于对数组元素的访问。将数组的首地址赋值给指令中的形式地址D，使变址寄存器n的值按顺序变化，就可以对数组中的各元素进行相同的操作。

Intel x86中采用变址寻址的指令举例：

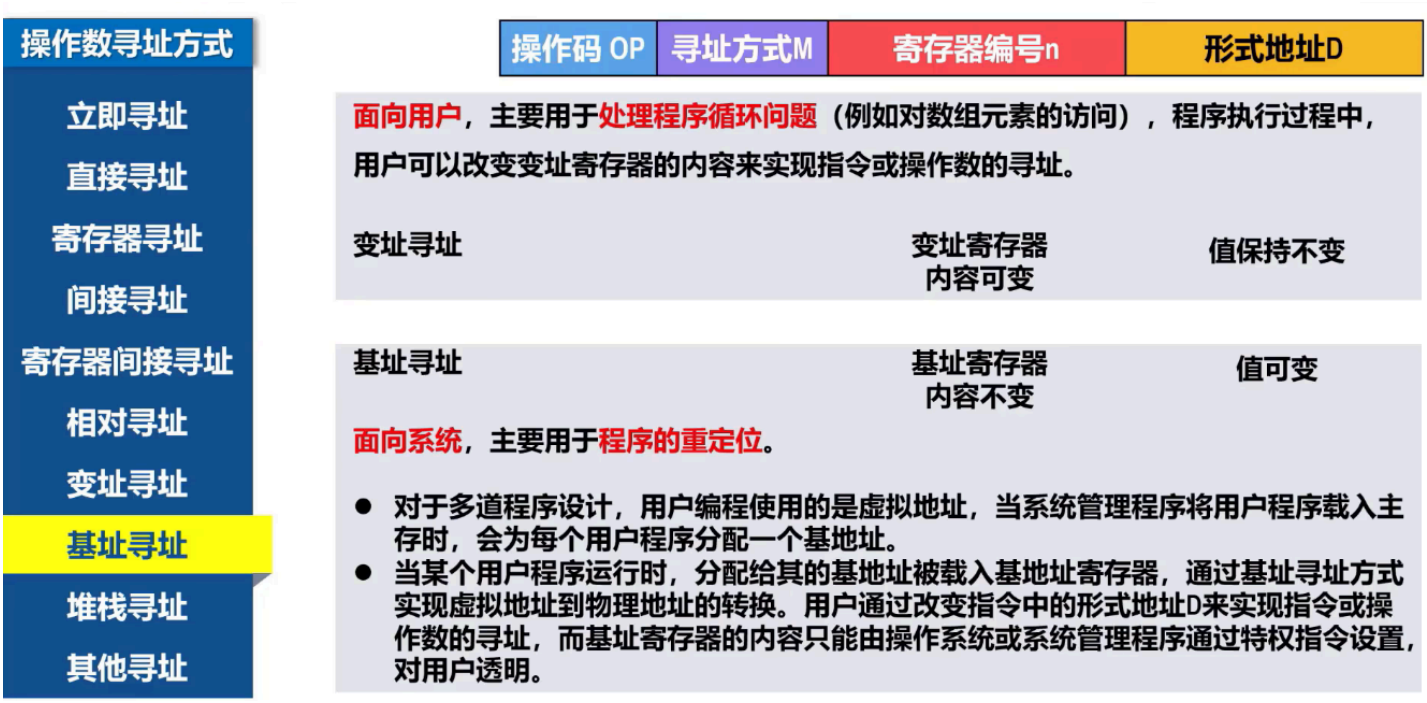
```
MOV  EAX, [ ESI + 32 ]  ；将32位通用寄存器ESI中的值加上偏移量32来形成主存地址，
                        ；再将主存地址所指向主存单元中的内容送入32位通用寄存器EAX
```

- 变址寄存器位数通常大于 D，可扩大寻址范围
- 主要用于数组元素访问：首地址放入 D，变址寄存器放“下标 × 元素大小”，循环中递增
- 例：`MOV EAX, [ESI+32]` 中 ESI 可变、32 固定

数组公式

$$\text{元素地址} = \text{首地址} + \text{下标} \times \text{元素大小}$$

1.8 基址寻址



四行卡

- **本质**：基址寄存器 BR 的内容保持不变，形式地址 D 可变，适合程序重定位
- **EA**：$EA = (BR) + D$
- **访存次数**：1（取操作数）
- **易错**：基址寄存器由操作系统/管理程序设定，用户程序不能随意修改（408 考试口径）；与变址寻址角色相反

补充

- 多道程序设计中，用户编程使用虚地址；程序装入主存时由系统分配基址，运行时 $\text{物理地址} = \text{基址} + \text{虚地址}$
- 基址寻址面向系统，主要用于程序重定位；变址寻址面向用户，主要用于数组/循环
- 真实 x86 中基址寄存器可由程序改写，注意与 408 考试口径区分

1.9 堆栈寻址



- 对存放在堆栈中的操作数进行寻址的方法称为堆栈寻址。
- 堆栈一般分为以下两种：
  - 寄存器堆栈
    - ✓ 在专用寄存器组中划分出来的一块、按“后进先出”（LIFO）规则管理的存储区，也称为硬堆栈。
    - ✓ 成本较高，不适合做大容量堆栈，较少使用。
  - 存储器堆栈
    - ✓ 在主存空间中划分出来的一块、按“后进先出”（LIFO）规则管理的存储区，也称为软堆栈。
    - ✓ 目前计算机普遍采用。
    - ✓ 由于存储器堆栈是在主存中划分出来的一块特定存储区，而主存基于地址进行访问，因此，需要使用一个特定的寄存器来记录堆栈中存储单元的地址，该寄存器称为堆栈指针（SP）。

四行卡

- **本质**：操作数在栈顶，SP 指向栈顶，按 LIFO 规则访问
- **EA**：$EA = (SP)$（软堆栈在主存）
- **访存次数**：1（软堆栈）；硬堆栈在寄存器组中则为 0
- **易错**：SP 指向栈顶元素还是栈顶空单元、栈向高地址还是低地址增长，都要看题设约定

补充

- 硬堆栈：在寄存器组中划分，成本高、容量小，较少使用
- 软堆栈：在主存中划分，容量大，现代计算机普遍采用
- 零地址指令的操作数通常隐含在栈顶/次栈顶

举例

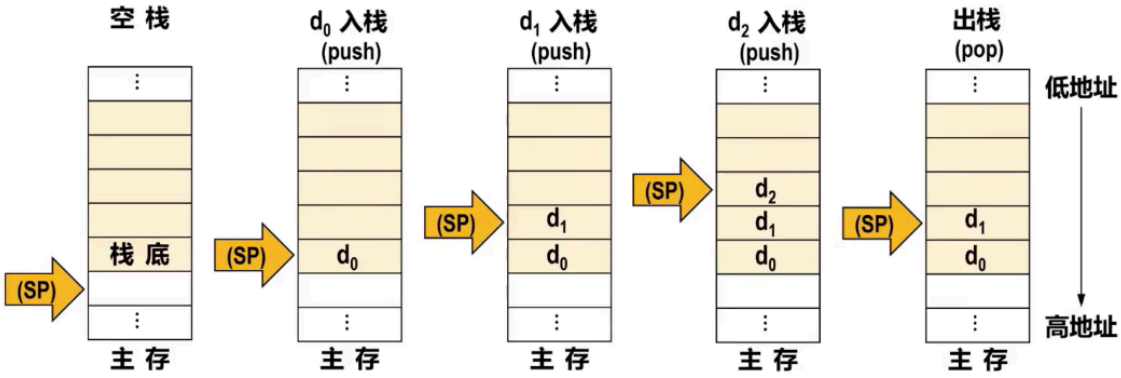
操作数寻址方式

- 立即寻址
- 直接寻址
- 寄存器寻址
- 间接寻址
- 寄存器间接寻址
- 相对寻址
- 变址寻址
- 基址寻址
- **堆栈寻址**
- 其他寻址

□ 存储器堆栈

- ✓ 在主存空间中划分出来的一块、按“后进先出”（LIFO）规则管理的存储区，也称为软堆栈。
- ✓ 由于存储器堆栈是在主存中划分出来的一块特定存储区，而主存基于地址进行访问，因此，需要使用一个特定的寄存器来记录堆栈中存储单元的地址，该寄存器称为堆栈指针（SP）。

【举例】某计算机中，堆栈指针SP指向栈顶单元，栈向下（即低地址）增长，主存按字节编址，栈操作以字节为单位。



- 本图约定：SP 指向栈顶元素，栈向低地址增长
- 进栈：先 SP-1，再存入数据；出栈：先取数据，再 SP+1
- 不同教材约定可能不同，做题以题设为准

1.10 复合寻址

操作数寻址方式

- 立即寻址
- 直接寻址
- 寄存器寻址
- 间接寻址
- 寄存器间接寻址
- 相对寻址
- 变址寻址
- 基址寻址
- 堆栈寻址
- **其他寻址**

- **将之前介绍的一些寻址方式进行组合，得到复合寻址方式。**
- **复合寻址方式主要用于复杂指令集计算机CISC。**

□ 相对寻址 + 间接寻址

❶ (PC) + D　相对寻址，得到存放操作数有效地址的主存单元的地址

❷ ((PC) + D)　间接寻址，得到操作数的有效地址

□ 变址寻址 + 间接寻址

❶ (变址寄存器) + D　变址寻址，得到存放操作数有效地址的主存单元的地址

❷ ((变址寄存器) + D)　间接寻址，得到操作数的有效地址

□ 间接寻址 + 变址寻址

❶ (D)　间接寻址，得到用于变址寻址的偏移量

❷ (变址寄存器) + (D)　变址寻址，得到操作数的有效地址

四行卡

- **本质**：多种寻址方式串联，先算内层 EA，再作为外层地址
- **EA**：按组合，如 $((PC)+D)$、$((IX)+D)$、$(IX)+(D)$
- **访存次数**：按组合累加，常见组合为 2 次
- **易错**：括号从内向外读；不要把“先变址后间址”理解成 $(IX+D)$

常见组合

- 相对 + 间接：$EA = ((PC) + D)$
- 变址 + 间接：$EA = ((IX) + D)$
- 间接 + 变址：$EA = (IX) + (D)$

注意

- **不同指令系统，可能采用了我们所介绍的寻址方式中的一部分。**
- **我们所介绍的操作数寻址方式，都是单地址指令。对于多地址指令，每个地址字段的作用不同，因此每个地址字段都可能有各自的寻址方式（寻址特征）字段。**

- 不同指令系统可能只采用上述寻址方式的一部分
- 多地址指令的每个地址字段都可能有各自的寻址方式字段，不要以为整条指令只有一个 M 字段

二、操作数寻址方式总结

指令格式

操作码 OP	寻址方式 M	形式地址 D

或

操作码 OP	寻址方式 M	寄存器编号	形式地址 D

	操作数 寻址方式	操作数的有效地址（主存地址）EA 程序计数器PC，形式地址D，通用寄存器N， 变址寄存器I，基址寄存器B	获取操作数需要的 访存次数 （不包括取指令）
	立即寻址	D为操作数本身	0
	直接寻址	$EA = D$	1
	寄存器寻址	$EA = N$	0
	（单级）间接寻址	$EA = (D)$	2
	寄存器间接寻址	$EA = (N)$	1
偏移寻址	相对寻址	$EA = (PC) + D$	1
偏移寻址	变址寻址	$EA = (I) + D$	1
偏移寻址	基址寻址	$EA = (B) + D$	1
复合寻址	相对寻址+间接寻址	$EA = ((PC) + D)$	2
复合寻址	变址寻址+间接寻址	$EA = ((I) + D)$	2
复合寻址	间接寻址+变址寻址	$EA = (I) + (D)$	2

寻址方式	EA	操作数访存次数（不含取指）	典型用途	核心易错
立即寻址	无，D 即操作数	0	常量、初值	D 是数不是地址
直接寻址	$EA = D$	1	访问固定主存单元	D 位数限制范围
寄存器寻址	$EA = R_D$	0	寄存器间运算	EA 是编号不是内容
间接寻址	$EA = (D)$	2（多级更多）	扩大寻址范围	D 不是 EA

寻址方式	EA	操作数访存次数（不含取指）	典型用途	核心易错
寄存器间接	$EA=(R_D)$	1	指针访问	与寄存器寻址差一次访存
相对寻址	$EA=(PC)+D$	1	位置无关代码、转移	PC 用更新后的值
变址寻址	$EA=(IX)+D$	1	数组遍历	IX 变、D 不变
基址寻址	$EA=(BR)+D$	1	程序重定位	BR 不变、D 变
堆栈寻址	$EA=(SP)$	1（硬堆栈 0）	现场保护、零地址	SP 指向约定
复合寻址	按组合，如 $((PC)+D)$	2	CISC 复杂寻址	括号由内向外

易错总结

- 访存次数不含取指；题目若问“总访存次数”，要加上取指次数
- 寄存器寻址与寄存器间接寻址只差一次访存，是选择题高频陷阱
- 相对寻址的 PC 基准、变址与基址的“谁变谁不变”、间接寻址的括号层数，是三大易错点
- 立即寻址和寄存器寻址的 EA 容易混：前者没有 EA，后者 EA 是编号

费曼自测

1. 哪些寻址方式取操作数不需要访问主存？→ 立即寻址、寄存器寻址（0 次）
2. 变址与基址的核心区别？→ 变址 IX 变、D 不变，面向用户数组；基址 BR 不变、D 变，面向系统重定位
3. 相对寻址的 PC 基准是什么？→ 取指后已更新的 PC，即下一条指令地址
4. 寄存器寻址与寄存器间接寻址的 EA 分别是什么？→ 寄存器编号 vs 寄存器内容
5. 复合寻址的 EA 计算顺序？→ 由内向外，先算括号内

知识关联：M/D 字段（5.4 寻址方式概述）；指令寻址（5.5 寻址方式——指令寻址）；EA 计算进入数据通路（6.5 数据通路）；基址与虚地址转换（4.31 虚拟存储器——概念和工作原理）；数组访问与变址（5.7 寻址方式——习题课）